Multi-Piano: LAN Multiplayer Terminal Piano

CMPE487

Ömer Faruk Erzurumluoğlu Muhammet Berkay Keskin

Department of Computer Engineering

June 2026

Boğaziçi University

Sections

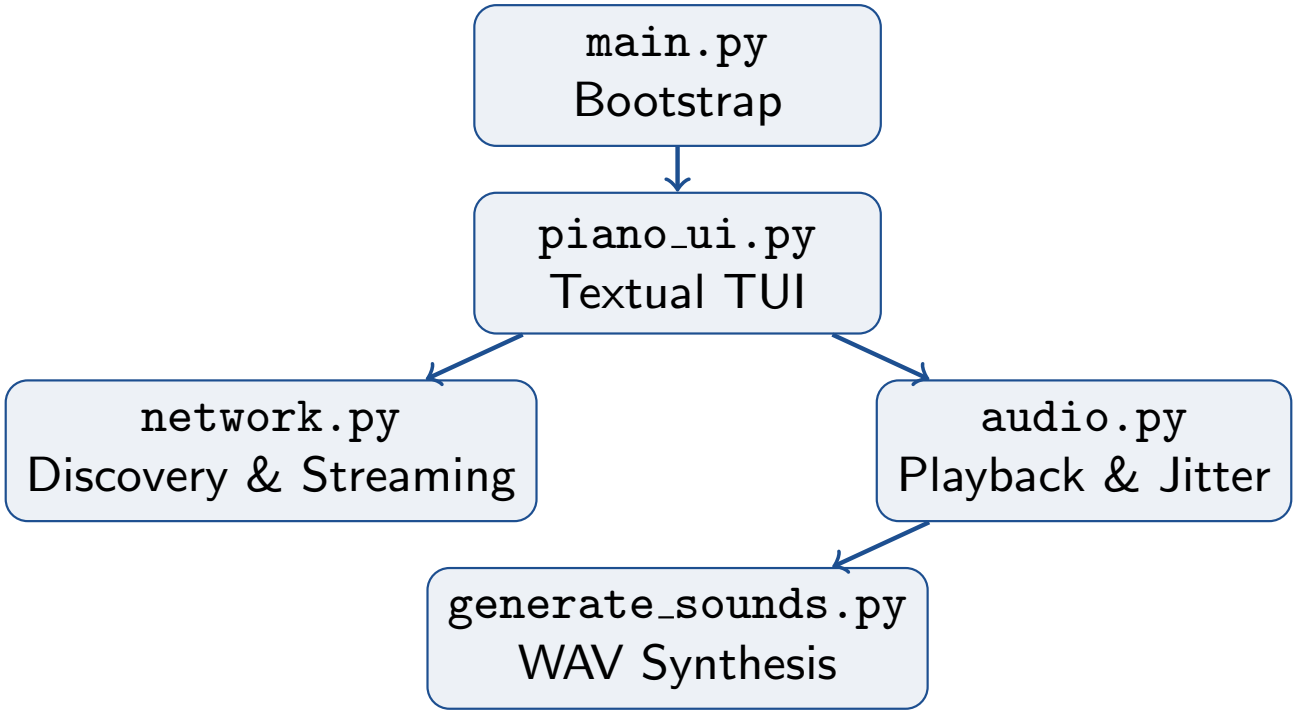
Project Overview

Multi-Piano is a LAN-based collaborative piano application with a terminal user interface (TUI).

- Multiple players on the same network discover, host, and join lobbies.
- Notes are transmitted in near real time and played back with synchronized audio.
- No external sound assets: piano tones are synthesized procedurally at build or first run.

Goal: demonstrate a lightweight multiplayer prototype combining discovery, mixed protocols, jitter-buffered audio, and a full TUI.

System Architecture



Networking — Three Channels

We separate concerns into three network channels:

Port	Protocol	Format	Role
12488	UDP	JSON	Lobby discovery (broadcast)
12487	TCP	length-prefixed JSON	Lobby management
12489	UDP	15-byte binary struct	Real-time note events

- **Discovery** finds open sessions without prior configuration.
- **TCP** maintains player membership and state synchronization.
- **UDP notes** minimize latency for press/release events.

Lobby Discovery (UDP 12488)

Client behaviour:

- Binds UDP port 12488 with `SO_BROADCAST` and `SO_REUSEADDR`.
- Sends `DISCOVER` broadcasts; listens for `LOBBY_REPLY` responses.

Host behaviour:

- Replies to discoverers with lobby name, host IP, TCP port, and active player count.
- Re-broadcasts `LOBBY_REPLY` every 2 seconds.

Stale lobby eviction: entries expire after 6 seconds without a reply (cleanup every 3 s).

Lobby Management (TCP 12487)

Join flow:

1. Client connects to `(host_ip, 12487)`.
2. Client sends `{"type": "JOIN", "SENDER_NAME": ..., "SENDER_IP": ...}`.
3. Host accepts, stores the socket, and appends the client IP to `peer_ips`.
4. Host broadcasts `STATE_UPDATE` with the current player list to all clients.

Disconnect is detected via TCP EOF; the lobby list is updated and the UI is notified.

Audio — Playback & Jitter Buffer

AudioEngine (pygame mixer):

- 32 channels; per-note volume scaled by `velocity / 100 * 0.8`.
- 200 ms fadeout on note release; watchdog stops notes held longer than 3 s.
- Graceful fallback: app continues silently if mixer init fails.

JitterBuffer for remote notes:

- Fixed **30 ms playout delay** from packet timestamp.
- Drops packets older than 500 ms on arrival.
- Sequence-number deduplication handles UDP duplicates.

Limitations & Future Work

Current limitations:

- Hub topology: guests hear the host, but **not each other**.
- LAN-only discovery; no Internet/WAN support or NAT traversal.
- Fixed 500 ms auto-release approximates note-off rather than true key-up.
- Single octave (13 keys); no sustain pedal or velocity from key pressure.

Possible extensions:

- Full-mesh or host-relay note forwarding for guest-to-guest audio.
- Wider keyboard range, MIDI input, and configurable instrument sounds.

Thank you for your attention

Ömer Faruk Erzurumluoğlu
Muhammet Berkay Keskin

omer.erzurumluoglu@std.bogazici.edu.tr
muhammet.keskin@std.bogazici.edu.tr

Boğaziçi University